Architecture

How it works and why it's built this way.

For: Everyone

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

This is the **hermes + langgraph** version. People chat with an assistant (Hermes Agent). The assistant decides which tool to call. The tools talk to policy-api, which runs each check as a **LangGraph graph** that saves its progress to Postgres after every step. When a check ends with "maybe" sections, it can pause until a person decides them in the chat. Other versions of the product are described in the README.

The job

When a head policy changes, find every policy in a chosen group that now conflicts with it, repeats its old wording, points to a section that moved, or uses a defined term whose meaning changed. Missing an affected policy is the expensive mistake, so the design favours **recall** (finding everything) over precision (flagging nothing extra), and it puts a person in the loop for anything uncertain.

The core rule

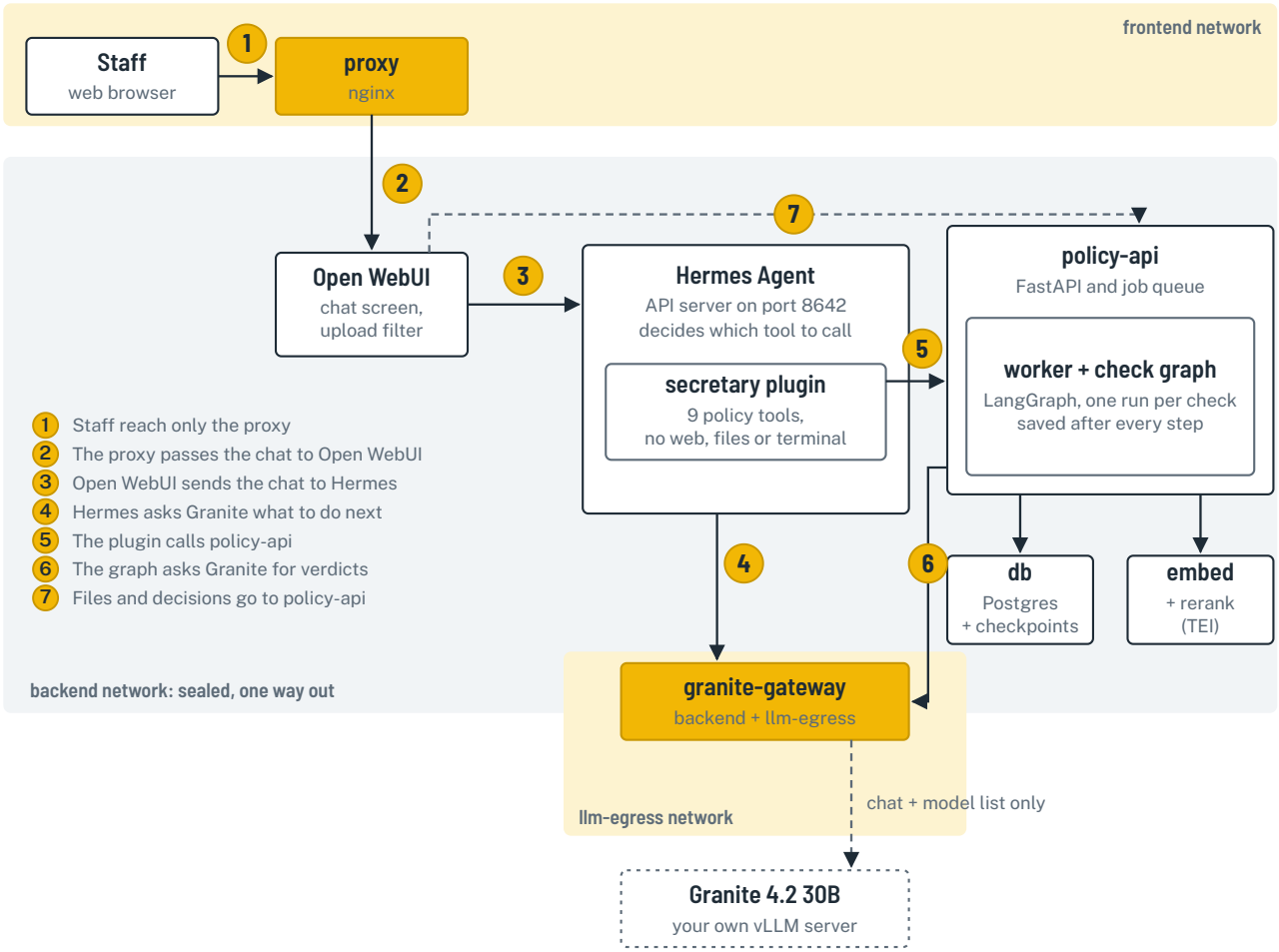
The chat model never decides impact itself. Hermes works out what the person is asking, calls a tool, and passes on the result. The graph inside policy-api makes every decision, and code builds every number, table and spreadsheet. The chat answer and the Excel file therefore always agree, and every decision can be traced in the audit trail.

Two more rules follow from it:

- **The graph's route is chosen by code, never by a model.** Models only answer one question about one pair of sections. They never pick the next step.
- **The model cannot record a review decision.** There is no tool for it. A decision exists only when a person types it, as their own message, and trusted code (the upload filter) reads it.

Hermes also has no other source of facts. It has no web search, no files, no terminal, no memory and no MCP servers. If the tools do not return it, the assistant says "Our policies don't cover that".

The parts in one picture



The request path. People reach only the proxy; Hermes and the check graph reach Granite only through the gateway.

Read the numbers 1 to 7 in the legend. Solid lines are requests; dashed lines are attached files, typed review decisions and the single way out to the Granite server.

Request flow, step by step

A. A question about what a policy says

Example: "What do we require for lockout devices?"

1. The person types in Open WebUI, reached through the `proxy`.
2. Open WebUI sends the chat to Hermes as an OpenAI-style request for the model `policy-assistant`, signed with `HERMES_API_KEY`.
3. Hermes builds a prompt from its rules (`hermes/SOUL.md`), the list of tools it may use, and the chat so far. It sends the prompt to Granite through `granite-gateway`.
4. Granite replies with a tool call, here `search_policies(question=...)`.

5. The plugin calls `GET /search` on policy-api. policy-api embeds the question (TEI `embed`), finds the nearest sections in Postgres (pgvector), and lets the reranker put them in order. The best 8 (up to 20) come back, each cut to 1,500 characters.
6. The plugin wraps each section in `<<<` and `>>>` so the model treats it as wording, never as instructions, and hands the text to Hermes. Hermes sends it to Granite.
7. Granite writes the answer and cites each fact as `[policy §section]`. Hermes returns it to Open WebUI.

If nothing matches, the tool says so and the assistant answers that the policies do not cover it. To read a whole policy or one section, Granite calls `read_policy` (`GET /policies/{id}/text`).

B. A question with an attached document

1. The person attaches a file. Before the message leaves Open WebUI, the **upload filter** (`openwebui/upload_filter.py`, a global function) sends the file to `POST /uploads` on policy-api with `POLICY_API_KEY`.
2. policy-api reads the file in its own limited process, splits it into numbered sections, stores it, and guesses which stored policy it is a new version of.
3. The filter adds a note to the end of the person's message: `[Attached file ...: upload id ...]`. It also stops Open WebUI from cutting the file into its own search index.
4. Hermes sees the note. Its rules tell it to pass the upload id to the tools.

C. A policy change check, end to end

Example: "Which Safety policies are affected by the change to POL-007?" with the new version attached.

1. Granite calls `check_policy_change(policy_group, head_policy, upload_id)`.
2. The plugin finds the group (`GET /subsets`) and the head policy (`GET /policies`). If a name is not a clear match, the tool returns a question for the person instead of guessing.
3. The plugin sends `POST /runs`, recording the request as made by `assistant`. policy-api stores a `runs` row and a row in the `jobs` table, and answers **202 Accepted** with a run id.
4. A worker inside policy-api claims the job (`FOR UPDATE SKIP LOCKED`) with a 60-second lease and renews the lease while it works. It starts the **graph** for that run (next section). The graph's state is saved to Postgres after each node. Many Granite calls run at once (`JUDGE_CONCURRENCY`, default 16).
5. While the job runs, the plugin asks `GET /runs/{id}` every 3 seconds. The Hermes turn waits, so the person's chat shows a running answer.
6. When the run is `done`, policy-api returns the chat answer (built in code) and a signed Excel link that works for 72 hours. The plugin passes both to Hermes with an instruction to show them exactly, and a note for Hermes alone with the 8-character check number.
7. Granite relays the answer. If the person then asks for a PDF, Word file or charts, Granite calls `make_check_files`, which uses `POST /runs/{id}/files`.

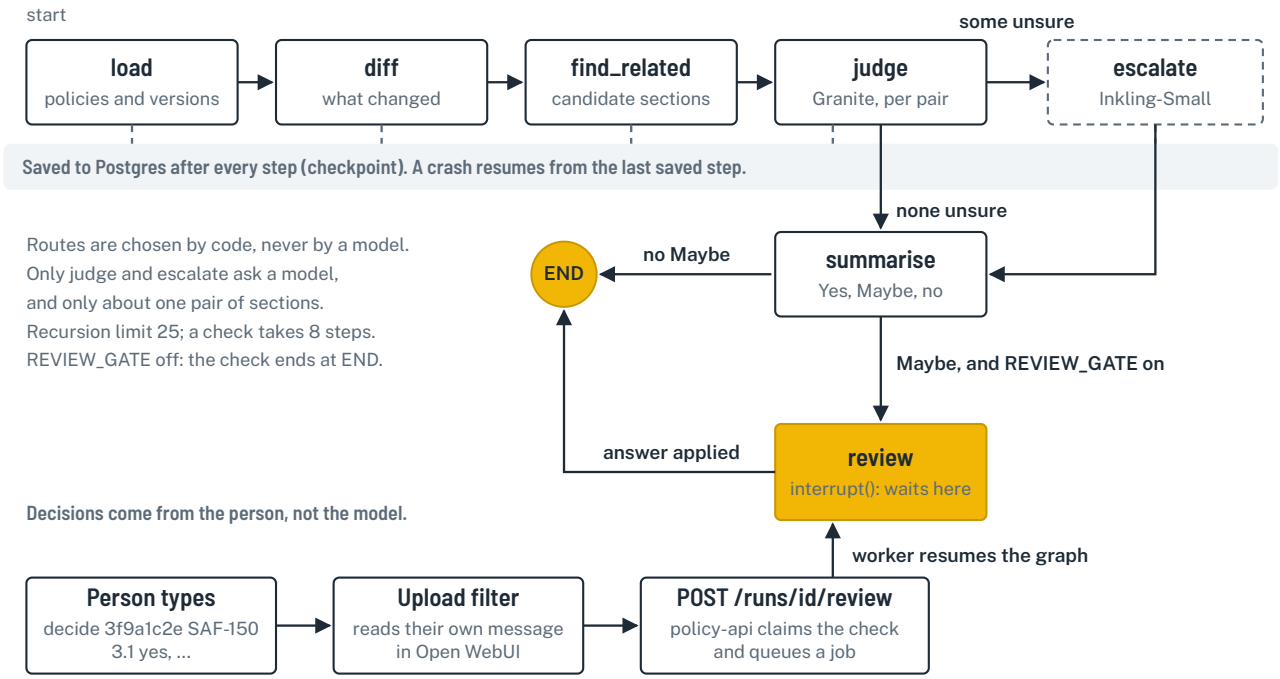
D. A check that pauses for a person

This happens only when `REVIEW_GATE=1` and the answer has at least one "maybe" section.

1. After `summarise`, the graph reaches the `review` node and calls LangGraph's `interrupt()`. The worker records the run as `needs_review`, with the list of "maybe" sections, and the job ends. The check now holds no worker and no lease; only its saved state waits in Postgres.
2. The plugin's next poll sees `needs_review`. It returns guidance to Hermes instead of an answer: show the list, show the check number (the first 8 characters of the run id), and tell the person to type a message that starts with `decide`, in the format below. The `<yes|no>` markers are placeholders the person replaces with their own answer.
3. The person types their whole message, for example `decide 3f9a1c2e SAF-150 3.1 yes, SAF-151 2.4 no`. (`decide 3f9a1c2e all no` answers every listed section.)
4. In Open WebUI, **before** the message goes to Hermes, the upload filter reads it. It accepts it only if the person's own latest message starts with the word `decide`. The filter takes the person's name from Open WebUI and calls `POST /runs/{check}/review` with `POLICY_API_KEY`.
5. policy-api checks that every listed section is really waiting, then **claims** the run with an atomic `needs_review → queued` update, so two answers sent together cannot both apply. It queues a `review_run` job and answers 202. A short check number is accepted here only among paused checks.
6. The filter adds a note to the person's message ("decisions were recorded", or why they were not). Hermes tells the person in plain words and calls `get_check_result`.
7. A worker runs the `review_run` job. It resumes the graph at the same thread with the person's answer. `review` turns each decided "maybe" into "Yes" (with "Confirmed by ") or drops it, recounts, and the graph ends. The run becomes `done` and the plugin returns the final answer.

Sections the person leaves out stay "Maybe". The check still ends, so it cannot be decided a second time. The decision changes the result of that check only. It is not written to the `judgements` rows used for training (review-and-training.md).

The check graph



One check as a graph. Code picks every route, a checkpoint is saved after each step, and a person's typed decision resumes a paused check.

Each check is one run of the graph in `policy_api/app/graph.py`. The graph is fixed. It has seven nodes, and code chooses every route.

Node	What it does	Uses a model?
`load`	Resolves the head policy, the two versions to compare (stored versions, or stored plus the upload) and the group's other policies. Stops with a plain reason if there is nothing to check	No
`diff`	Clause-level comparison of old and new head policy: modified, added, removed and renumbered sections, and defined terms touched. Saves each change	No
`find_related`	For each changed section, collects clauses in the group that cite the head policy, use a changed defined term, or are similar in meaning (embedding search, then the reranker keeps the top N)	TEI only
`judge`	Granite reads each (changed section, related clause) pair and returns yes / no / unsure, a reason, evidence quotes and a suggested edit, forced into a JSON schema	Granite
`escalate`	"Unsure" pairs go to Inkling-Small for a second opinion. Skipped if no escalation model is set up	Inkling-Small
`summarise`	Per policy: Yes if any section is "yes", Maybe if any is "unsure", otherwise not affected. Builds the result	No
`review`	Pauses (`interrupt`) until a person decides the "maybe" sections, then applies the answer	No

The `find_related` node keeps two kinds of clause that the reranker never cuts: those that cite the head policy and those that use a changed defined term. Explicit links are where most real impact is. The similarity search catches policies that overlap in meaning without citing the head policy.

Conditional edges. Three decisions, all made in code:

After	Goes to	When
`judge`	`escalate`	At least one pair is "unsure" and an escalation model is configured
`judge`	`summarise`	Otherwise
`summarise`	`review`	At least one row is "Maybe" and `REVIEW_GATE` is on
`summarise`	end	Otherwise

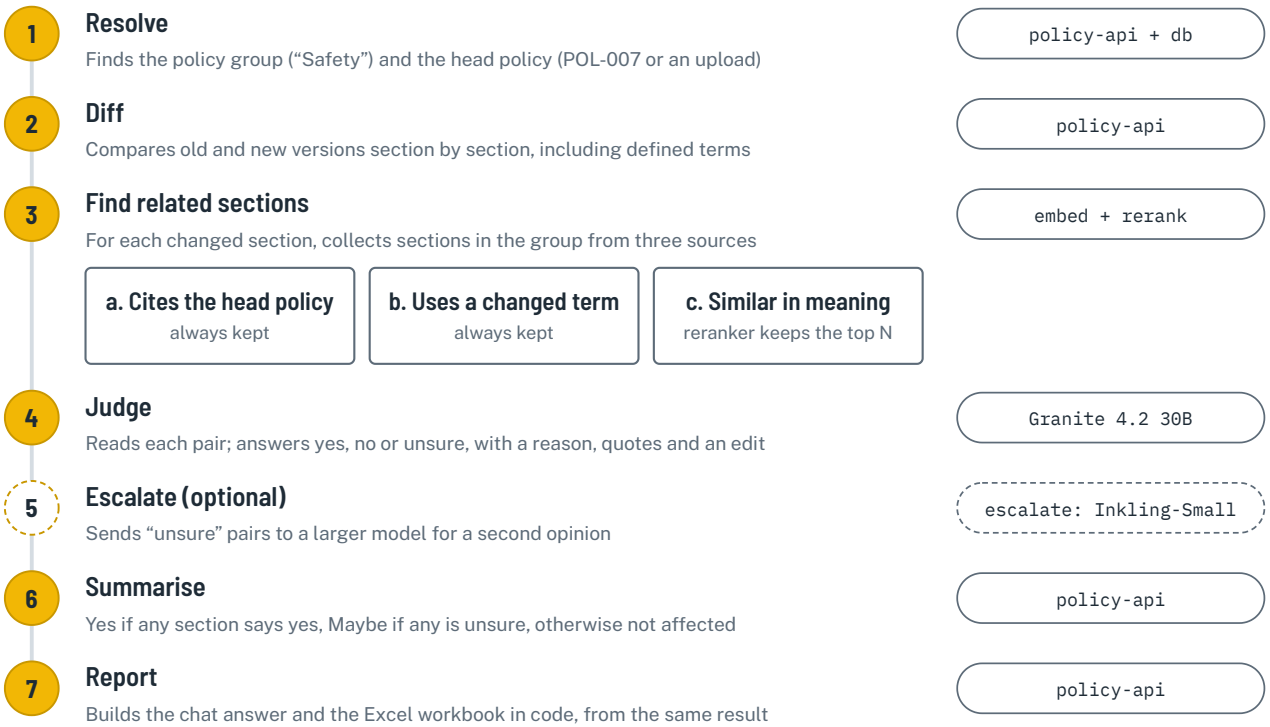
State (`RunState`). Plain JSON values only, so a saved checkpoint never depends on the code's classes.

Field	Holds
`run_id`	The run this graph belongs to (also the checkpoint thread id)
`head`, `labels`, `policy_ids`, `metas`	The head policy, the names of the two versions compared, the group's other policies and their details
`changes`	The changed sections, each with the id of its saved row
`pairs`	One entry per (change, related clause): the clause, how it was found, the verdict, and whether it was escalated
`errors`	How many pairs the model could not judge
`result`	The finished answer: rows, counts, not-affected list
`reviewed`	The person's answer, once the check has been decided

Checkpointer. `AsyncPostgresSaver` (langgraph-checkpoint-postgres 3.2.0, LangGraph 1.2.16) writes the state to the same Postgres database after every node (`durability="sync"`, so the write finishes before the next node starts). It creates its own tables on first start. The thread id is the run id. In demo mode and in tests it uses memory instead.

Recursion limit. 25 steps. A normal check takes at most 8, so reaching the limit means a bug, not a big check. The check then fails with "Something went wrong on our side" and the error is logged.

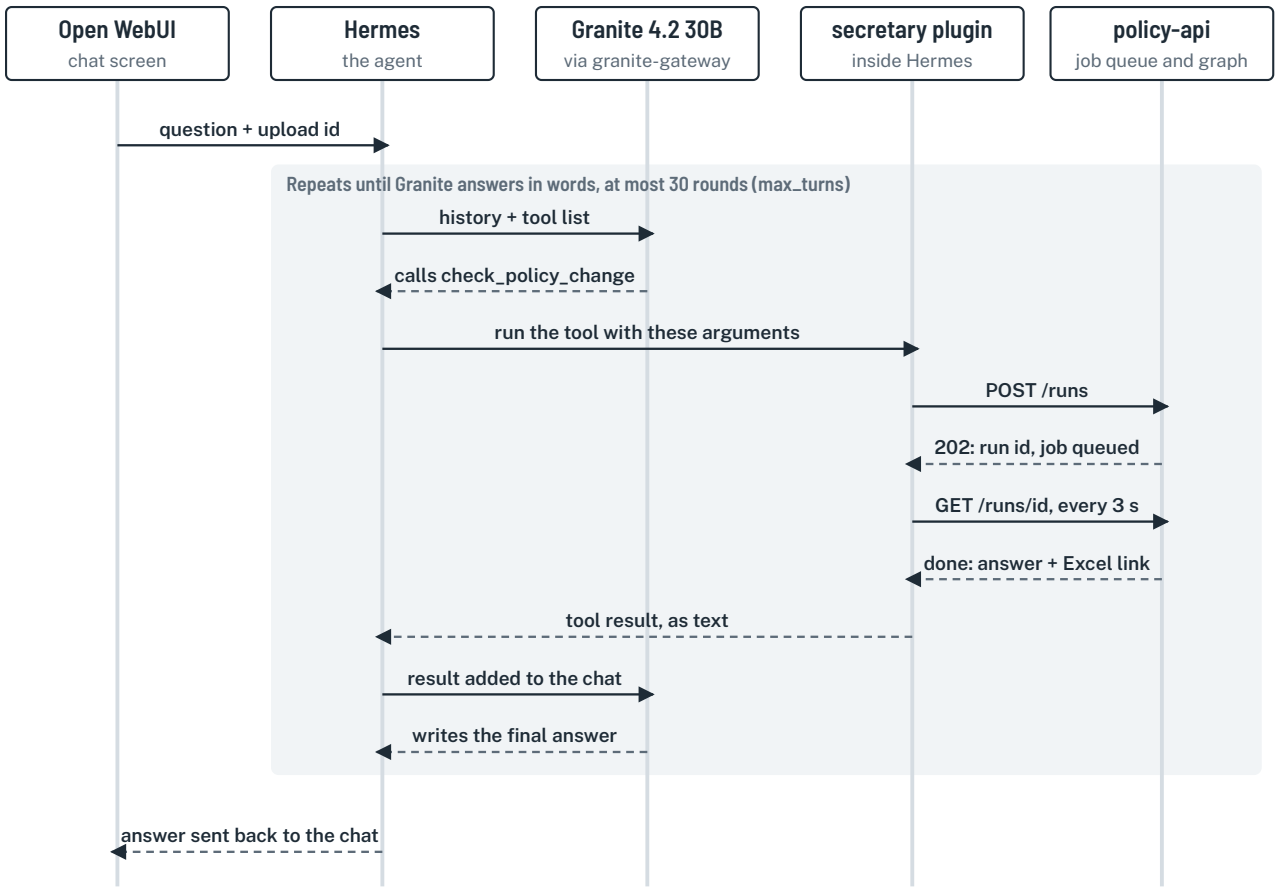
Resume. When a job is retried, the worker asks LangGraph for the run's saved state. If steps are left, the graph carries on from the last finished node. Only the node that was running is repeated, and its half-written verdicts are deleted first.



How one question is answered. Code makes every decision; the judge rules on one pair at a time.

The same steps in plain order. Steps 1 to 6 are the graph nodes in the table above (step 1 is `load`). Step 7 happens after the graph: the chat answer and Excel file are built from the saved result when someone asks for them.

The agent loop



One question, step by step. Granite proposes tool calls; the plugin runs them against policy-api; code makes every decision.

Hermes repeats the same small loop for every question: ask Granite, run the tool Granite picks, give Granite the result, and ask again. The loop ends when Granite answers in words instead of asking for a tool. Hermes stops it after `max_turns: 30` rounds.

The plugin returns plain text, not data. Each result starts with an instruction to Granite ("show this answer exactly as written", "answer only from these sections"). That is how numbers, policy IDs and links reach the person unchanged.

Components

Component	Image and version	Role	Networks
`proxy`	`nginx:1.27-alpine` (`NGINX_TAG`)	The only thing people reach: Open WebUI, plus report downloads and `/health`. Holds no data	backend + frontend
`open-webui`	`ghcr.io/open-webui/open-webui` (`OPENWEBUI_TAG`, pinned in `.env`)	Chat screen. Sends chats to Hermes. Runs the upload filter (files and review decisions). Trimmed: no web search, tools, memories or code running	backend

Component	Image and version	Role	Networks
`hermes`	`nousresearch/hermes-agent` (`HERMES_TAG`; tested with v0.22.3)	The assistant. OpenAI-compatible API on port 8642, model name `policy-assistant`. Runs the agent loop and the `secretary` plugin	backend
`policy-api`	Built from `policy_api/` (Python 3.12, FastAPI 0.115 or later; `APP_VERSION`). Uses LangGraph 1.2.16 and langgraph-checkpoint-postgres 3.2.0	The check graph, the job queue and worker, search, uploads, reports and downloads	backend
`db`	`pgvector/pgvector:pg17` (`PGVECTOR_TAG`)	Postgres with pgvector: policies, clauses, graph, runs, jobs, audit trail, and the graph's checkpoints	backend
`embed`, `rerank`	`text-embeddings-inference` (`TEI_TAG`) with Granite embedding and reranker models	Embeddings for similarity search; reranker to order and cut candidates	backend
`granite-gateway`	`nginx:1.27-alpine`	The backend's only way out. Passes chat and model-list calls to `GRANITE_URL`, adds the Granite key, checks TLS	backend + llm-egress
Granite 4.2 30B	Your own vLLM server, outside this stack (vLLM 0.20 or later)	The model behind Hermes' reasoning and the graph's verdicts	Reached only through the gateway
`escalate` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), Inkling-Small	Second opinion on "unsure" pairs	backend
`extract` (optional)	`vllm/vllm-openai` (`VLLM_TAG`), gpt-oss-20b	Finds references the regex misses during ingest	backend
Review and training services (optional)	Argilla v2.8.0 and its helpers; `lora-trainer`; `judge-candidate`; `judge-tuned`	Review queue and fine-tuning (review-and-training.md)	backend

The LangGraph graph runs **inside** `policy-api`. It is not a separate container.

The Open WebUI tool file `openwebui/policy_impact_tool.py` is **not used** in this version. Nothing in `compose.yaml` or the setup steps installs it, and Open WebUI's tool servers are switched off. Hermes and its plugin do that job. The file stays in the repo for the `main` and `langgraph` versions. The **upload filter** is different: it is required here, for files and for review decisions.

Hermes: settings and why

The settings are in `hermes/config.yaml`, mounted read-only, so nobody can change them from the chat.

Setting	Value	Why
Model	`granite-4.2-30b` through `http://granite-gateway:8000/v1`	One model, one way out. The gateway adds the key, so Hermes holds none

Setting	Value	Why
`context_length`	131072	What the Granite server serves. Hermes refuses a model with less than 64,000 tokens, so the server needs `--max-model-len` of **65536 or more** (131072 recommended)
Tools allowed (`api_server`)	`secretary`, `todo`, `delegation`	The policy tools, a to-do list for multi-part requests, and subagents for side tasks
Tools switched off	web, search, browser, vision, image and video, speech, terminal, file, code execution, skills, memory, session search, scheduled jobs, kanban, connections	Answers come only from the policy database. Nothing can reach outside or change a file
`tool_search`	off	Hermes would otherwise hide some tools behind a search step. A mid-size model handles a short, plain list of tools more reliably
`max_turns`	30	Hermes' default is no limit. A loop that never ends would never free the chat
Subagents (`delegation`)	Same model, 20 steps each, 3 at a time, no subagents of their own	Keeps side tasks small
Helper calls (titles, compression, approvals)	Sent to the main model	"Auto" would try outside providers first. The sealed network would block them, but slowly
Memory, user profile, curator, background review	Off	Nothing is remembered between chats, and the assistant writes no skills of its own
Update checks, model catalog, lazy installs	Off	No calls home, nothing installed at run time
Plugin	`secretary` only	The tools below

Open WebUI also has its own title, tag and follow-up generation switched off. Each one would be a full Hermes run.

The plugin's tools

The plugin is `hermes/plugins/secretary/`. Every tool calls policy-api with `POLICY_API_KEY`. Each tool returns text with instructions for the model, and turns network errors into a plain message.

Tool	What it does	Calls
`search_policies`	Finds sections that answer a question, optionally within one group or one policy	`GET /search`
`read_policy`	Reads a whole policy, or one section with its subsections, at a chosen version	`GET /policies` (to resolve the name), `GET /policies/{id}/text`
`list_policy_groups`	Lists groups such as Safety or HR	`GET /subsets`
`find_policy`	Finds a policy by ID or title words	`GET /policies`

Tool	What it does	Calls
`check_policy_change`	Starts a check and waits for the result. If the check pauses, returns the guidance for the person	`GET /subsets`, `GET /policies`, `POST /runs`, then `GET /runs/{id}`
`get_check_result`	Gets the result of a check that was still running or paused, by check number	`GET /runs/{id}`
`make_check_files`	PDF, Word, Excel or charts of a finished check	`GET /runs/{id}`, `POST /runs/{id}/files`
`compare_policies`	Compares two policies, two versions, or a policy and an attached document	`GET /policies`, `POST /compare`
`make_document`	Word, PDF, Excel or chart from content the model writes, for requests that are not a check	`POST /documents`

There is **no tool for deciding "maybe" sections**. A decision comes only from the person's own typed message, through the upload filter (flow D). Nothing the assistant or a tool writes can count as one.

Data stores

Store	Holds	Notes
Postgres (`pgdata` volume)	Policies and every version, clauses with embeddings, the reference graph, groups, uploads, runs, the job queue, every model verdict, and the graph's checkpoint tables	Back up daily (operations.md). Checkpoints are not deleted when a check ends
`exports` volume	Excel, PDF, Word and chart files	Removed after `EXPORT_RETENTION_DAYS` (default 30)
`hermes-data` volume	Hermes' own working folder	Config, rules and plugin are mounted read-only on top. Memory is off
`openwebui` volume	Users, chats, the upload filter and its key	Back up with `.env`
Argilla databases (optional)	Review queue	Separate from the policy database

Network and security boundaries

```
frontend network:   people → proxy
backend network:    proxy → open-webui → hermes → policy-api → db, embed, rerank
                    open-webui (upload filter) → policy-api    (files, review decisions)
backend + egress:   hermes and policy-api → granite-gateway → Granite server
```

- `backend` is `internal: true`: no route out. `proxy` is the only container on `frontend`. The gateway is the only container on `llm-egress`.
- The gateway passes `/v1/chat/completions` and `/v1/models` and nothing else, and checks the Granite server's certificate.

- Hermes has no route to the internet and no tool that would use one. It reaches policy-api and the gateway, and nothing else.
- policy-api answers only requests with the right key and a known `Host` name. The proxy shows people only Open WebUI, report downloads and `/health`.
- The 8-character check number works only on `GET /runs/{id}` and `POST /runs/{id}/review`, and a review looks it up only among paused checks. Every other route needs the full id.

Three keys keep the parts apart. Each is held by as few parts as possible.

Key	Where it lives	Who uses it	What it protects
`POLICY_API_KEY`	`.env`	policy-api (to check it); Hermes (as `SECRETARY_API_KEY`, for the plugin); the upload filter (a setting in Open WebUI)	Every policy-api route except signed downloads
`HERMES_API_KEY` (16+ characters)	`.env`	Hermes (as `API_SERVER_KEY`); Open WebUI (as `OPENAI_API_KEY`)	Hermes' chat API
`GRANITE_API_KEY`	`.env`	`granite-gateway` only	The Granite server. Hermes and policy-api never see it

`LLM_API_KEY` guards the optional local models. `LINK_SIGNING_KEY` signs download links. See security.md.

When things fail

What fails	What happens
Granite server down or slow	Chat: Hermes gets no reply from Granite, so the chat shows an error. Checks: in `judge`, each pair that fails is marked "could not be checked automatically" and shows as Maybe, for a person to check. If **every** pair fails, the check fails at once with "The checking model is not responding" and is not retried
Second model (Inkling-Small) fails in `escalate`	The first opinion is kept
policy-api down	Each tool returns "The policy service did not respond"; Hermes tells the person to try again later. A check already queued waits in the database
Worker or policy-api crashes in the middle of a graph	The 60-second lease runs out (or, after a restart, the worker takes its own jobs back at once). Any worker picks the job up and the graph **carries on from the last saved node**. Only the node that was running repeats
A temporary error inside a node (database blip, model restart)	The job is retried up to `JOB_MAX_ATTEMPTS` (default 3), with growing waits (up to 5 minutes, random), each time from the last saved node. After that the job is `dead` and the person sees "failed several times in a row"
A problem with the question (no such policy, nothing changed)	Not retried. The person gets a plain reason
The graph runs past its recursion limit (25)	Treated as an unexpected error: retried, then the check fails with "Something went wrong on our side" and the error is logged

What fails	What happens
A check takes longer than 10 minutes (`SECRETARY_MAX_WAIT_MINUTES`)	The plugin stops waiting and tells Hermes the check is still running. The check itself continues. The person can ask again and Hermes calls `get_check_result` . It must not start a second check
Open WebUI waits longer than 15 minutes (`OPENWEBUI_TIMEOUT_SECONDS` , 900)	The chat shows an error. Keep the plugin's wait below this so it cannot happen. The check still finishes and can be fetched by check number
Hermes loops without answering	Hermes stops after 30 rounds (`max_turns`)
A paused check is never answered	It stays `needs_review` for as long as it takes. There is no time limit, no reminder and no worker is tied up. Asking for the result repeats the request for a decision. The saved state stays in Postgres
A decision is typed wrongly	The filter or policy-api refuses it and says why in a note; the check stays paused. A section not on the list is refused (422); a check that is no longer paused is refused (409)
The upload filter is off or not Global	Files are not sent, and decisions cannot be recorded. The check stays paused
A file cannot be read	The upload filter puts a plain note in the message; Hermes tells the person

Limits

- **No per-person identity reaches policy-api through Hermes.** Open WebUI knows who is signed in, but Hermes does not pass that on. Every check is stored as requested by `assistant` . Use Open WebUI's own account records to see who asked, and tell staff their questions are logged. The name on a review decision is the exception: the upload filter reads it from Open WebUI itself.
- **Check numbers are not tied to the person who asked.** Anyone signed in who has the 8-character number of a paused check can decide it. The number is a convenience, not a password (security.md).
- **Everyone with an account can ask about every group.** There is no per-user group filter.
- **Training still targets gpt-oss-20b.** The fine-tuning scripts and the `judge-candidate` and `judge-tuned` services are set up for that model, not for Granite. Verdicts can be sent to a tuned judge; the chat stays on Granite (review-and-training.md). Decisions typed in the chat are not training data.
- **The Granite server must be set up for this.** Tool calling, structured output and a context of at least 65536 tokens (setup.md).
- **A very long check holds the chat open.** The plugin waits up to 10 minutes (`SECRETARY_MAX_WAIT_MINUTES`); Open WebUI waits 15 (`OPENWEBUI_TIMEOUT_SECONDS` , 900). Keep the first below the second, and set any proxy in front of the stack to at least the same. If a check runs longer, Hermes says so and the result is fetched later by check number.
- **A partly answered review ends the check.** Sections left out stay "Maybe" and cannot be decided later.
- **Hermes follows its rules by instruction, not by force.** It cannot reach the internet or change a file, because those tools are switched off. It can still word an answer badly. That is why numbers, links and verdicts are produced by code and passed through unchanged. Its rules say not to show check numbers unless asked, except when a person must copy one to decide.

Why these choices

Choice	Why	Instead of
Hermes picks the tool; a fixed graph decides impact	The assistant handles the conversation (which group, which policy, which file, follow-up questions). The graph is deterministic, testable and auditable	An agent that plans its own steps and judges impact itself (more tokens, results vary between runs, harder to audit)
LangGraph for the check, with Postgres checkpoints	A check of hundreds of model calls can survive a crash and carry on from the last node. The same mechanism gives a clean pause for a person (`interrupt`) with no new service	Starting over after every crash, or a workflow engine such as Temporal as a new stateful system
Routing decided in code	The same input takes the same route every time, so results can be audited	Letting a model choose the next node
Review decisions read by the upload filter	The filter is trusted code that sees only the person's own message. The model has no decide tool, so it cannot approve anything, even if a policy contains hidden instructions	A decide tool for Hermes (the model could be talked into using it)
A plugin that returns text with instructions, and nothing else	Hermes has no web, files, terminal or memory, so a wrong answer can be badly worded but cannot leak or change anything	Giving the agent general tools and trusting it to behave
Granite 4.2 30B as judge and chat model	US-developed (IBM), Apache 2.0, dense 30B that fits one 80 GB GPU in bf16, 128K context, tool calling, thinking can be switched off per request for fast, repeatable verdicts	Inkling for everything (needs 4–8 high-end GPUs per replica)
Use the Granite server you already run, behind one gateway	No second copy of a 58 GB model to host and update. One narrow, checked way out (two routes, one address, key held in one place) instead of a model container inside the stack	Starting a local judge container, or letting each service call the server on its own
Inkling-Small only for "unsure"	Larger-model judgement where it matters, at a fraction of the cost	Running every pair through the large model
Reranker before judging	Cuts hundreds of similar clauses to about 20 per change; the biggest compute saving	Judging every similar clause
JSON-schema guided decoding	Every verdict is valid and machine-readable	Asking for JSON in the prompt and hoping
Postgres for the graph of policies	Recursive queries cover policy hierarchies; one fewer system to run	A separate graph database
Excel built with openpyxl	Exact, repeatable, safe from formula injection	Letting the model write the spreadsheet
Job queue in Postgres (`FOR UPDATE SKIP LOCKED`, leases)	Runs survive restarts with no new service to run or secure; Postgres is already here	Temporal or Dagster: another stateful system to operate inside the contained network, for a queue of a few hundred jobs a day

Data model (db/init.sql)

- `policies`, `policy_versions`, `clauses`: every version is kept. `current_clauses` is a view of each policy's latest version.
- `refs`: the reference graph (which clause cites which policy and section), for latest versions.
- `subsets`: which policies belong to which group. A policy can be in several groups.
- `uploads`, `runs`, `changes`: each question and what changed. A run's status is `queued`, `running`, `needs_review`, `done` or `failed`.
- `judgements`: one row per model decision, with model name, the exact prompt and its hash, verdict, and the reviewer's decision. This is the audit trail and the training data.
- `jobs`: the work queue (see operations.md). `schema_migrations` records which files in `policy_api/app/migrations/` have been applied; `db/init.sql` is the starting point for a new database.
- The checkpoint tables are made by LangGraph's Postgres saver on first start. They are not in `db/init.sql`.

Clause parsing rules

A new clause starts at a line that begins with a section number:

- a Markdown heading with a number (`## 4 Lockout devices`)
- any multi-level number (`4.2 Devices are inspected…`, `4.2.1) …`)
- a single number followed by a short title (`4 Lockout devices`)

`1. Do this.` inside a clause stays part of that clause. Text before the first number is clause `0`. Policy IDs are found with `POLICY_ID_PATTERN` (default `[A-Z]{2,6}-\d{1,5}`), in forms like `POL-007 §4.2`, `§4.2 of POL-007`, `POL-007 section 4` and `POL-007` alone (whole policy).

Roadmap

Built since v0.1: the job queue (operations.md), the review queue and LoRA fine-tuning (review-and-training.md), and reports, comparisons and charts (how-to.md).

1. **OCR** for scanned PDFs, and reading Word's automatic numbering (see known limits in how-to.md).
2. **Kubernetes** manifests when the models need more than one node.